Aerial Robotics Kharagpur Documentation Task 2.2 – Image Cleaning and Denoising

Parv Chopra *

***Abstract*—This report documents the solutions to Tasks 2.2.1 and 2.2.2, both centred on image processing and noise removal. Task 2.2.1 addresses cleaning a noisy binary image of Iron Man: after an initial attempt using a custom 5×5 weighted-average kernel with Canny edge removal and a line prediction pipeline, the final approach uses Otsu's binarisation with connected-component cluster pruning to produce a clean binary image. Task 2.2.2 tackles denoising a heavily corrupted colour image: a comprehensive noise analysis characterises the corruption as additive, independent, Gaussian noise with $\sigma \approx 63$, after which five standard denoising algorithms (Gaussian blur, bilateral filter, non-local means, channel-wise YCrCb denoising, and BM3D) were tried as initial attempts, and a custom BFS-based edge-aware blur was developed as the final approach. These tasks demonstrate classical image-processing techniques applicable to perception pipelines in aerial robotics and computer vision.**

I. Introduction

Task 2.2 comprises two sub-tasks in the domain of image processing:

- **Task 2.2.1 – Image Cleaning:** Given a noisy grayscale image (`iron_man_noisy.jpg`), clean it to recover the underlying structure as a clear binary image.
- **Task 2.2.2 – Noise Analysis & Image Denoising:** Given a heavily corrupted colour image (`noisy.jpg`), characterise the noise statistically and apply multiple denoising algorithms to recover the original image.

The challenge in both tasks is balancing noise removal with structure preservation — aggressive filtering removes noise but also destroys thin lines and fine details, while gentle filtering leaves residual noise that degrades downstream processing.

II. Problem Statement

A. Task 2.2.1

Given a noisy grayscale image:

1) **Clean** the image to produce a clear binary representation of the underlying lines/shapes.

B. Task 2.2.2

Given a noisy colour image:

1) **Analyse** the noise to determine its type, distribution, standard deviation, and spectral characteristics.
2) **Denoise** the image using multiple methods.
3) **Compare** the denoising methods and identify the most effective approach.
4) **Implement** a custom edge-aware denoising method that preserves structural edges.

III. Related Work

Gaussian/Median filtering are standard noise-removal approaches. Gaussian filtering smooths uniformly; median filtering preserves edges better but can remove thin structures [1].

Otsu's thresholding [1] automatically selects a binarisation threshold by minimising intra-class variance, and is effective when the histogram is bimodal.

Connected-component analysis labels contiguous regions; it is useful for removing small noise clusters by area [1].

Hough Line Transform [1] maps edge points to (ρ, θ) parameter space to detect lines. The probabilistic variant (`HoughLinesP`) detects line segments directly.

Bilateral Filter [1] is an edge-preserving filter that weights pixels by both spatial distance and colour similarity.

Non-Local Means [1] compares patches across the entire image and averages similar ones, providing excellent detail preservation for Gaussian noise.

BM3D groups similar patches into 3D arrays and applies transform-domain collaborative filtering. It is widely considered the best classical denoiser.

IV. Initial Attempts

A. Task 2.2.1 – Image Cleaning

1) Method 1 – Custom Kernel Filtering + Canny Edge Removal: A hand-designed 5×5 weighted-average kernel was applied to smooth the image. The kernel emphasises neighbouring pixels while de-weighting the centre pixel (centre weight $= 0$):

$$K = \frac{1}{25} \begin{bmatrix} 0 & 1 & 1 & 1 & 0 \\ 1 & 1.25 & 2 & 1.25 & 1 \\ 1 & 2 & 0 & 2 & 1 \\ 1 & 1.25 & 2 & 1.25 & 1 \\ 0 & 1 & 1 & 1 & 0 \end{bmatrix} \tag{1}$$

The zero centre weight means the output at each pixel depends entirely on its neighbours, providing aggressive smoothing. This was followed by binary thresholding at intensity 90, and Canny edge detection whose detected edges were removed (set to 0) to clean up boundary artefacts from the thresholding step.

Result: Produced a clean binary image but lost thin line details due to the aggressive smoothing kernel.

2) Attempt 2 – Line Prediction using Hough Transform: After cleaning with Method 1, a line prediction pipeline was attempted. Canny edge detection (thresholds 50, 150) extracted edge pixels, and `cv2.HoughLinesP` (threshold 50, min length 30 px, max gap 10 px) detected line segments. Each segment was extended to the image boundaries, and morphological dilation (5×5, 2 iterations) followed by closing (5×5, 2 iterations) attempted to fill gaps. A morphological thinning algorithm was then applied to iteratively peel away boundary pixels until only single-pixel-wide skeletons remained.

Result: The pipeline detected 192 line segments and produced filled/thinned outputs, but the results were unsatisfactory — the aggressive smoothing from Method 1 combined with line extension introduced artefacts, and the morphological gap filling distorted the original structure rather than recovering it.

B. Task 2.2.2 – Noise Analysis

Before denoising, a comprehensive noise analysis was performed (`examine.py`):

1) **Global per-channel statistics**: Mean and standard deviation of each B, G, R channel across the entire image.
2) **Flat-patch noise estimation**: Automatically detected the flattest 32×32 region (lowest local standard deviation) and measured noise σ in that patch.
3) **Channel correlation**: Computed inter-channel Pearson correlation coefficients of the noise residuals.
4) **Distribution analysis**: Fitted a Gaussian distribution to the noise histogram.
5) **Additive vs. multiplicative test**: Compared noise σ in bright vs. dark regions. Constant σ indicates additive noise; σ scaling with intensity indicates multiplicative noise.
6) **Frequency analysis (FFT)**: Computed the power spectrum of the noise. A flat spectrum confirms white noise.

Key findings:

- Noise is **additive** (constant σ regardless of region brightness).
- Noise is **independent** across channels (low inter-channel correlation).
- Noise is approximately **Gaussian** (good histogram fit, skewness ≈ 0, kurtosis ≈ 0).
- $\sigma \approx 63$ per channel (very heavy).
- Noise is **spectrally white** (flat FFT).

Conclusion: The noise model is i.i.d. Gaussian $\mathcal{N}(0, 63)$ added independently to each R, G, B channel — the classic AWGN model.

C. Task 2.2.2 – Trial Denoising Methods

Five standard denoising methods were tried before developing the final custom approach:

1) Method A – Gaussian Blur: A simple low-pass filter: a 21×21 box blur followed by two passes of 7×7 Gaussian blur. Very fast but blurs edges. Used as a baseline.

2) Method B – Bilateral Filter: Edge-preserving filter with parameters $d = 9$, $\sigma_{\text{color}} = 75$, $\sigma_{\text{space}} = 75$. Weights pixels by both spatial distance and colour similarity. Better edge preservation than Gaussian blur but slower.

3) Method C – Non-Local Means: Patch-based denoising: `cv2.fastNlMeansDenoisingColored` with $h = 30$, template window 5×5, search window 21×21. Compares patches across a search area and averages similar ones. Followed by a light 3×3 Gaussian blur to smooth residual artefacts.

4) Method D – YCrCb Channel-wise Denoising: Converts to YCrCb colour space to exploit human perceptual sensitivity:

- **Y** (luminance): denoised lightly ($h = 6$) to preserve detail.
- **Cr, Cb** (chrominance): denoised aggressively ($h = 15$) since the human eye is less sensitive to colour noise.

Each channel is processed independently with `fastNlMeansDenoising`, then merged and converted back to BGR.

5) Method E – BM3D: Block-Matching and 3D Filtering: the state-of-the-art classical denoiser. The image is normalised to $[0, 1]$, and BM3D is applied per channel with $\sigma_{\text{psd}} = 63/255 \approx 0.25$. Groups similar patches into 3D arrays and applies transform-domain shrinkage.

Result: While BM3D and NL-Means produced strong results numerically, none of these methods provided strict edge preservation — they all blur edges to some degree. This motivated the development of the custom edge-aware blur as the final approach.

V. Final Approach

A. Task 2.2.1 – Image Cleaning

The final approach uses Otsu's binarisation followed by connected-component cluster removal (`final.py`):

Step 1 – Otsu's Automatic Thresholding: The noisy grayscale image is binarised using Otsu's method [1]. Otsu's algorithm works by examining the histogram of pixel intensities and finding the threshold value that minimises the weighted sum of variances within the two classes (foreground and background). For every possible threshold $t \in [0, 255]$, it splits the pixels into two groups — those below t and those above — computes the variance within each group, and selects the t that minimises the combined intra-class variance:

$$t^* = \arg\min_t \left[\omega_0(t)\,\sigma_0^2(t) + \omega_1(t)\,\sigma_1^2(t)\right] \tag{2}$$

where ω_0, ω_1 are the class probabilities and σ_0^2, σ_1^2 are the class variances. This automatically adapts to the image's histogram without manual threshold tuning.

Step 2 – Connected-Component Analysis: All contiguous regions are labelled using 8-connectivity (including diagonal neighbours).

Step 3 – Small Cluster Removal: Any connected component with fewer than 7 pixels is removed (set to 0). This eliminates isolated noise speckles — which are typically small

and disconnected — while preserving genuine line segments and shapes, which form larger contiguous regions.

Output: `method2_cleaned.jpg` — a clean binary image of the Iron Man drawing with noise removed and structure preserved.

Fig. 1. Task 2.2.1 final output: cleaned binary image after Otsu binarisation and cluster removal.

B. Task 2.2.2 – Custom Edge-Aware Blur

The final approach for denoising is a fully custom edge-aware blur implementation (`try.py`) consisting of:

1) **Edge detection**: Multi-scale Sobel gradients with non-maximum suppression and hysteresis thresholding — a complete edge detector implemented from scratch without using `cv2.Canny`.
2) **BFS flood-fill blur**: For each non-edge pixel, BFS explores a kernel-sized window. The flood *cannot cross edge pixels*, so only reachable non-edge pixels on the same side of the edge contribute to the average. This ensures edges are never blurred across.
3) **Edge pixel inpainting**: Edge pixels are iteratively filled from their non-edge neighbours (outside-in), preventing black edge artefacts in the output.

Key parameters:

Parameter	Default	Description
`EDGE_PRE_BLUR`	15	Gaussian blur before edges
`SOBEL_KSIZE`	9	Sobel kernel size
`LOW_PCT`	75	Lower hysteresis percentile
`HIGH_PCT`	85	Upper hysteresis percentile
`EDGE_DILATE`	1	Edge dilation in pixels
`BLUR_KERNEL`	7	BFS blur window size
`BLUR_ITERS`	3	Number of blur passes

Fig. 2. Task 2.2.2 final output: edge-aware blur result.

VI. Results and Observation

A. Task 2.2.1

Two cleaning methods were compared. Method 1 (custom kernel + Canny) produced a binary image but lost thin line details due to the aggressive smoothing. The subsequent line prediction pipeline (Hough Transform + morphological gap filling + thinning) introduced further artefacts rather than improving the result.

The final approach — Otsu binarisation + connected-component cluster removal — produced significantly better results. Otsu's automatic threshold selection adapts to the image histogram, and the connected-component pruning targets isolated noise speckles specifically rather than blurring globally. The output (`method2_cleaned.jpg`) is a clean binary image preserving the underlying line structure.

B. Task 2.2.2

Table I summarises all denoising methods tried.

TABLE I
Comparison of denoising methods on a noisy image with $\sigma \approx 63$.

Method	Speed	Edges	Noise	Overall
A. Gaussian	Fast	Poor	Good	Baseline
B. Bilateral	Medium	Good	Medium	Better
C. NL-Means	Slow	Very Good	Very Good	Strong
D. YCrCb	Slow	Very Good	Very Good	Strong
E. BM3D	Slow	Excellent	Excellent	Strong
Custom BFS	**V. Slow**	**Excellent**	**Good**	**Final**

The custom BFS edge-aware blur was selected as the final approach because it provides strict edge preservation — edges are never blurred across, unlike all other methods which blur edges to varying degrees. While Methods A–E produced varying levels of noise removal, they all compromise edge sharpness. The BFS approach guarantees that the blur window cannot cross detected edges, making it the most structurally faithful denoiser despite its higher computational cost.

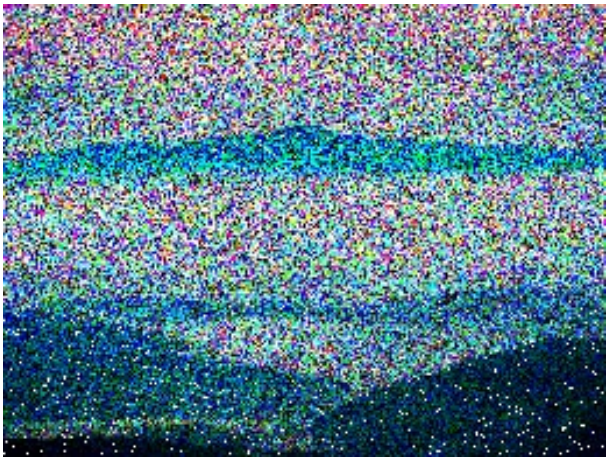

Fig. 3. Original noisy input image ($\sigma \approx 63$).

VII. Future Work

A. Task 2.2.1

- **Adaptive cluster-size threshold**: The minimum cluster size (currently fixed at 7) could be optimised per image based on noise statistics.
- **Multi-scale Otsu**: Applying Otsu at multiple resolutions and combining results could improve robustness for images with varying noise density.
- **Deep learning denoising**: A trained denoising autoencoder could outperform hand-crafted thresholding on complex noise patterns.

B. Task 2.2.2

- **Deep learning denoisers**: Trained models (DnCNN, FFDNet, Restormer) can outperform classical methods, especially at high noise levels.
- **Automatic noise-level estimation**: Automatic σ estimation from the image would make the pipeline fully automatic.
- **GPU acceleration**: The BFS-based edge-aware blur is extremely slow. A GPU implementation or approximation using guided filtering could achieve similar results orders of magnitude faster.
- **Blind denoising**: Methods that work without knowing σ (e.g. noise2noise, self2self) would be more practical in real-world scenarios.

Conclusion

This report presented solutions to two image-processing tasks. For Task 2.2.1, two cleaning approaches were tried — custom kernel filtering with Canny edge removal (followed by an unsuccessful line prediction pipeline), and Otsu's binarisation with connected-component cluster pruning. The latter was adopted as the final approach, providing clean noise removal while preserving the underlying line structure.

For Task 2.2.2, a comprehensive noise analysis pipeline confirmed the noise as additive, independent, Gaussian with $\sigma \approx 63$ and spectrally white. Five standard denoising methods (Gaussian blur, bilateral filter, non-local means, YCrCb channel-wise, and BM3D) were tried as initial approaches. The final solution is a custom edge-aware blur using BFS flood-fill, which guarantees strict edge preservation during denoising by preventing the blur window from crossing detected edges.

Acknowledgement: Rudraksh Gupta supported during the development and debugging of this project.

References

[1] OpenCV, "OpenCV Documentation," https://docs.opencv.org/, accessed 2025.